Chapter 7

GPPT: Graph Pre-training and Prompt Tuning to Generalize Graph Neural Networks

7.1 Motivation

Graph neural networks (GNNs) [33, 145] have become a prominent technique to analyze graph structured data in many real-world systems, including social networks [112], recommender systems [146], and knowledge graph [113]. The general approach of GNNs treats the input graph as an underlying computation graph, and learns the node representations by passing messages across the edges. The generated node representations could be used for different downstream tasks, such as link prediction [59, 95], node classification [45, 17], and biochemical module classification [147, 148].

Recent efforts in transfer learning have advanced GNNs to capture the transferable graph patterns and generalize to different downstream tasks [149, 150, 151]. Specifically, most of them follow the "pre-train, fine-tune" learning strategy: using the easily accessible information as pretext task (e.g., edge prediction) to pre-train GNNs, and fine-tuning over downstream task with the pre-trained model as initialization. By leveraging the substantial corpus of unlabeled structure, the pre-training has become an attractive solution for few-shot graph analysis [152, 153], where the labeled data is scarce. Under the "pre-train, fine-tune" framework, the existing works mainly focus on objective engineering: tailoring the pre-training objectives to better inform the downstream applications. The prevalent pre-training approaches include edge

prediction [154] and contrastive learning [149].

One of key limitations in "pre-train, fine-tune" is the training objective gap between the constructed pretext and dedicated downstream tasks. This gap would hinder us from directly and efficiently eliciting the pre-trained graph knowledge. We take the pretext edge prediction and downstream node classification as an example. Traditionally, the pre-trained GNNs were transferred by simply replacing the final classification layer, and fine-tuned together with the new parameters. While the pre-trained parameters optimize the binary edge predictions of node pairs, they need to be tuned time consumingly towards the parameter space conducting multi-class node predictions of standalone nodes, especially when the connected nodes have distinct classes. It is observed that such fine-tuning even results in worse downstream performance than training from scratch [154], meaning the negative and failing knowledge elicitation. The previous objective engineering often carefully designs the pretext task close to every new downstream application, which requires both expert knowledge and tedious manual trials.

Instead of being limited in the objective engineering, we propose to explore the novel strategy of "pre-train, prompt, fine-tune" to bridge the task gap in GNNs. The prompt technique is motivated by natural language processing (NLP) [155], where it is applied to reformulate the downstream task looking similar to the pretext one. We use the example of language model pre-trained with masked word prediction, and the downstream classification task with raw input text (e.g., emotion classification of "I missed the bus today"). The prompting function aims at modifying the input text to a prompt by appending the semantic textual template (e.g., "I missed the bus today. I felt so [MASK]"). In this way, the classification task is reformulated to fill the masked emotion word (e.g., "wonderful"), which is in line with the pre-training objective.

The pre-trained model could be applied with or without slight fine-tuning to quickly elicit the pre-trained knowledge. This unique advantage of efficient fine-tuning has improved the applicability of generally pre-trained model to a wide series of language tasks [156, 157, 158].

However, it is non-trivial to design the prompting function to generalize GNNs due to following two challenges. First, given the symbolic graph data, it is infeasible to apply the semantic prompting function to bridge the various graph machine learning tasks. One cannot use the textual template (e.g., "I felt so [MASK]" as adopted in NLP) to reformulate the node classification to edge prediction, which are defined to learn the abstract nodes instead of semantic texts. Second, even with the feasible prompting function, it is unaware how to design the informative prompt to better reformulate the downstream application. Using the example of node classification, the prompt should take into account the relative neighborhoods of target node to boost the classification accuracy. In view of these two challenges, we propose the research question: *how to design the graph-aware prompting function to bridge the pretext and downstream tasks in fine-tuning the pre-trained GNNs?*

Presented work. To mitigate the training objective gap, we present graph pre-training and prompt tuning (GPPT) framework. Notably, most of the graph pre-training learn the structure knowledge via edge prediction or contrastive learning to evaluate the similarity score of any instance pair, while the downstream application is node classification since the local node labels are scarce. Without loss of generality and following the previous efforts, we adopt the masked edge prediction, the most simplest and universal pretext task, to pre-train GNNs. Based on the pre-trained model, we propose the graph prompting function to reformulate the downstream node

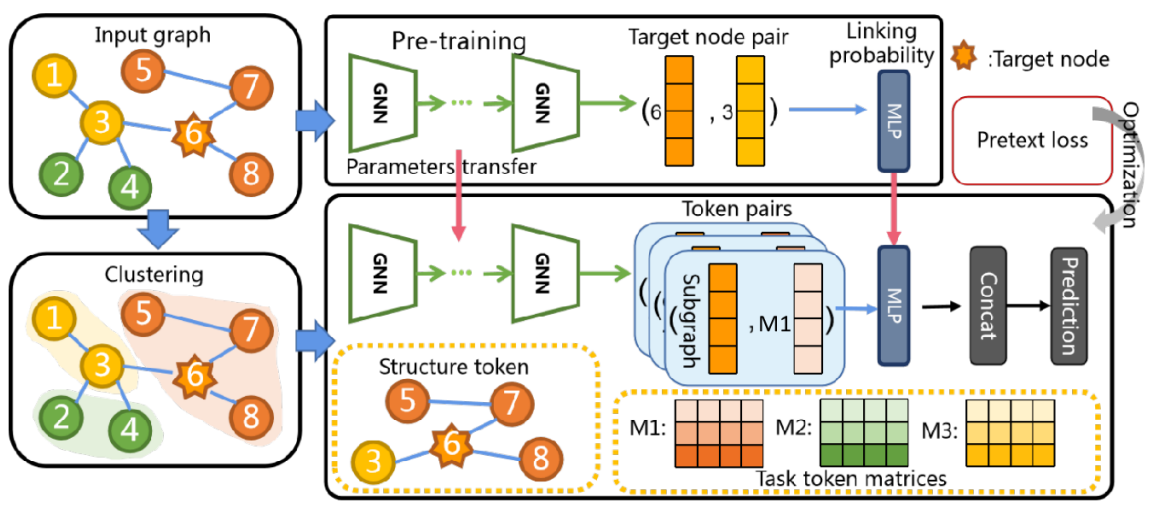

Figure 7.1 : Overview of GPPT. The masked edge prediction is leveraged to pre-train GNNs. To bridge training objective gap, the graph prompting function modifies the standalone node into token pairs, and directly applies the pre-trained model without changing classification layer (i.e., MLP). The token pair contains task token (denoting node label) and structure token (describing node). The node classification is reformulated to measure the linking score of token pair. The task token with the highest score is decided as node label.

classification problem looking the same as edge prediction. As shown in Figure 7.1, the graph prompting function applies the pairwise token template to modify the standalone node into token pairs: the task token representing the downstream problem and the structure token containing the node information. Specifically, they respectively tackle the above two challenges as below:

First, to unify as edge prediction (challenge #1), the task token represents a node label waiting to be classified with trainable continuous vector, and is appended to the target sample. We measure the linking probability between task token and target

sample directly through the pre-trained model. Given the multiple possible labels, the node classification is realized by choosing the task token/node label with the highest linking probability. Meanwhile, the orthogonal prompt initialization and regularization are proposed to separate the trainable vectors of different label tokens.

Second, to inform the node classification (challenge #2), the structure token represents the target sample with its multi-hop neighborhoods. The intuition is that nodes are prone to share the similar patterns with their proximal neighbors. The structure token would make it much easier to accurately classify a node by automatically selecting the relative neighbors.

Finally, we fine-tune the graph prompting function together with the pre-trained GNNs. We design extensive experiments on eight benchmark datasets and two downstream applications. The empirical results show that GPPT consistently outperforms all the other training strategies, including supervised learning, joint training and traditional transfer learning. In the few-shot graph analysis problem, GPPT delivers the average improvement of 4.29%, and saves the fine-tuning time cost up to 4.32X.

7.2 Preliminary

In this section, we introduce the preliminary of graph neural networks and the transfer learning scheme of "pre-train, fine-tune".

7.2.1 Graph Neural Networks

Let tuple $\mathcal{G} = (X, A)$ denote the undirected graph, where $X \in \mathbb{R}^{N \times d}$ is the node features and $A \in \mathbb{R}^{N \times N}$ is the adjacency matrix. Note that N and d denote the number of nodes and feature dimensions, respectively. Each node v_i is described by feature vector $x_i \in \mathbb{R}^d$, which is given by the i-th row in matrix X. The graph link is

denoted by $A_{ij} = 1$ if nodes v_i and v_j are connected; otherwise $A_{ij} = 0$.

Following the spatial message passing strategy [33], GNNs learn the node representation by recursively aggregating the representations of its neighbors. Formally, at the k-th layer, the representation learning of node v_i is:

$$x_i^{(k)} = \text{AGGREGATE}(\{x_j^{(k-1)}, v_j \in \mathcal{N}(v_i) \cup v_i\}, \theta^{(k)}). \tag{7.1}$$

$x_i^{(k)} \in \mathbb{R}^d$ is the embedding vector of node v_i learned at the k-th layer; and $x_i^{(0)}$ at the initial layer is given by x_i. $\mathcal{N}(v_i)$ is the set of first-order neighbors adjacent to node v; and $\theta^{(k)} \in \mathbb{R}^{d \times d}$ is the trainable weights. AGGREGATE denotes the aggregation function to pass the neighborhood embeddings to center node v_i, and then combines them (e.g., through sum, mean or max pooling) to generate the new node representation. Suppose the number of graph convolutional layers is K. To facilitate the following expression, we use $h_i = f_\theta(\mathcal{G}, v_i)$ to represent the final node representation learned from K-layer GNNs, where $\theta = \{\theta^{(1)}, \cdots, \theta^{(K)}\}$ denotes the concatenated trainable parameters. It has been widely demonstrated the optimized representation h_i delivers superior performance in many applications, such as edge prediction (e.g., item exploration in recommender system [159]) and node classification (e.g., topic categorization in scientific citation networks [160]).

7.2.2 Pre-train and Fine-tune GNNs

The supervised learning of node representation usually requires plenty of annotated data from each dedicated downstream task [161]. However, while it is resource expensive to manually tag the plentiful nodes and links in large-scale graphs, it is also inefficient to train GNNs from scratch for each downstream task. The transfer learning framework of "pre-train, fine-tune" has recently shown the potential to provide the

attractive solution [149, 150, 151]. The core of pre-training GNNs is to use easily-accessible information to encode the intrinsic graph structure. The pre-trained GNNs could serve as initialization to generalize other downstream applications.

There are several efforts proposed to construct the self-supervised pretext tasks to pre-train GNNs. To encode the intrinsic structure knowledge, masked edge prediction [145] and contrastive learning [162] are the most effective and popular pretext tasks in literature. While the masked edge prediction computes the linking probabilities of node pairs to generate edges, the contrastive learning compares the similarity scores of graph instance pairs. Actually, the contrastive learning could be understood from the view of edge prediction, where the positive graph instance pair should be assigned with the highest similarity (i.e., linking probability). Therefore, without loss of generality and to ease the understanding of graph prompt, we use the masked edge prediction as pretext task to introduce the pre-training. Our work could be easily extended to any other pretext task which optimizes the pairwise score loss.

The edge prediction pretext task works as follows: we first randomly mask partial edges and then train GNNs to reconstruct them. Formally, let $\mathcal{G}^{\text{pre}} = (X, A^{\text{pre}})$ denote the masked graph, where a set of edges is randomly sampled and masked to be $A_{ij} = 0$. Therefore, the node representation learning of node v_i is given by $h_i = f_\theta(\mathcal{G}^{\text{pre}}, v_i)$. The pretext task is to decide whether a node pair is connected, where GNNs is pre-trained to optimize following loss:

$$\min_{\theta,\phi} \sum_{(v_i,v_j)} \mathcal{L}^{\text{pre}}(p_\phi^{\text{pre}}(h_i, h_j); g(v_i, v_j)). \tag{7.2}$$

Node pair (v_i, v_j) is given by the masked edges, or sampled from the negative unconnected pairs. p_ϕ^{pre} is the projection head with trainable parameters ϕ to evaluate similarity score of node pair (v_i, v_j), such as multiple layer perceptron (MLP) as

shown in Figure 7.1. $\mathcal{L}^{\text{pre}}$ is the pretext loss function, such as cross entropy. $g(v_i, v_j)$ denotes the ground-truth label of node pair, which is indicator A_{ij} in our edge prediction pretext task. The optimized parameters θ^{pre} are expected to encode the graph connectivity structure and provide a good initialization for the downstream tasks.

The downstream application in graph data is often defined by the local-level node classification [154], e.g., classifying node label or attribute, where the ground truth is hard to be obtained. Under the traditional "pre-train, fine-tune" transfer learning framework, GNNs are fine-tuned to optimize the following loss:

$$\begin{aligned} \min_{\theta,\varphi} \quad & \textstyle\sum_{v_i} \mathcal{L}^{\text{down}}(p_{\varphi}^{\text{down}}(h_i); g(v_i)), \\ \text{s.t.} \quad & \theta^{\text{init}} = \theta^{\text{pre}}. \end{aligned} \tag{7.3}$$

The constraint means GNNs are initialized by the optimized parameters from pretext task. $p_{\varphi}^{\text{down}}(h_i)$ denotes the new projection head accompanied with parameters φ, while the pretext projection head is discarded. $\mathcal{L}^{\text{down}}$ is the downstream loss function (e.g., cross entropy), and $g(v_i)$ denotes the ground-truth label of node v_i.

7.3 Graph Prompting Framework

Due to the training objective gap existing between the pretext and downstream tasks, the pre-trained GNNs may fail to be efficiently leveraged by the new application, and even lead to negative transfer. First, we note that GNNs' parameters θ are optimized to generate close embeddings for connected node pairs, instead of nodes of the same class. If the disconnected pairs share the same class, the pre-trained model requires to be tuned with many epochs to adapted to the new problem. This time-consuming fine-tuning prevents us from efficiently using the pre-trained model. The pre-trained knowledge will also be gradually filtered out in the long tuning process. Second, considering parameters φ of the new projection head, they are hard to be coupled with

the pre-trained GNNs at the initial stage. Therefore, the downstream projection head tends to conduct wrong classifications. In the experiment part, we empirically show that the vanilla transfer learning in Eq. (7.3) even results in poor results, comparing with the plain GNNs without any pre-training.

In this section, we propose to bridge the training objective gap with prompt, which reformulates the downstream task looking the same as pretext task. We are inspired by the successful applications of prompt tuning in NLP, where all the downstream applications are unified according to the pre-trained language model [163]. The details of NLP prompt tuning are in the related work. We define the new transfer learning framework of "pre-train, prompt, fine-tune" for GNNs, and tailor the prompting function to graph data.

7.3.1 Pre-train, Prompt, Fine-tune

Instead of changing the projection head and classifying the standalone nodes, the graph prompt-based learning framework modifies the input node to token pair, which is applied directly to the pre-trained model. We lay out the general mathematical definition of graph prompting function as below, and then give the concrete prompting function to design the token pair.

Definition 7.3.1 (Graph prompting function)**.** *A graph prompting function* f_{prompt} *is applied to change the standalone node into prompt:* $v_i' = f_{\text{prompt}}(v_i)$*. Prompt* v_i' *has the same input shape to the pretext projection head.*

Definition 7.3.2 (Pairwise prompting function)**.** *Considering the edge prediction or contrastive learning pretext task, prompt* v_i' *is described by the template of token pairs:* $v_i' = f_{\text{prompt}}(v_i) = [T_{\text{task}}(y), T_{\text{srt}}(v_i)]$*.* $T_{\text{task}}(y)$ *is the task token to describe the downstream application, and* $T_{\text{srt}}(v_i)$ *is the structure token to represent target node* v_i*.*

Rethinking the node classification downstream task, the task token $T_{\text{task}}(y)$ could be given by a node label waiting to be classified, while the structure token $T_{\text{srt}}(v_i)$ could be represented by the subgraph surrounding target node v_i to provide more structural information. Given the token pairs $[T_{\text{task}}(y), T_{\text{srt}}(v_i)]$, by embedding them into continuous tensors, one is able to conduct the classification task by fitting the linking probability between the two tokens. GPPT shown in Figure 7.1, the new paradigm of "pre-train, prompt, fine-tune" for GNNs, contains three components:

Prompt addition. In this step, the graph prompting function generates a series of token pairs to be classified. Assuming that there are total C classes $[y_1, \cdots, y_C]$, we construct their corresponding token pairs $[T_{\text{task}}(y_c), T_{\text{srt}}(v_i)]$, for $c = 1, \cdots, C$.

Prompt answer. Given each token pair $[T_{\text{task}}(y_c), T_{\text{srt}}(v_i)]$, we embed them into continuous vectors. We then concatenate them as input to the pre-trained projection head, and obtain the linking probability. We answer and classify target node v_i with label y_c if it obtains the highest probability.

Prompt tuning. Following the pretext training objective, we optimize the following loss to fine-tune GNNs:

$$\min_{\theta,\phi} \sum_{(v_i,y_c)} \mathcal{L}^{\text{pre}}(p_\phi^{\text{pre}}(T_{\text{task}}(y_c), T_{\text{srt}}(v_i)); g(y_c, v_i)). \tag{7.4}$$

$g(y_c, v_i)$ denotes the ground truth connection between label class y_c and target node v_i. $g(y_c, v_i) = 1$ if node v_i belongs to class y_c; otherwise it is zero. Therefore, we have the same training objective (i.e., edge prediction) with the pretext task, and bridge the optimization objective gap. Below we introduce how to design the task and structure tokens tailored to graph data.

7.3.2 Graph Prompting Function Design

As defined in Definition 7.3.2, choosing an appropriate prompting function has great impacts on the learning processes of downstream task and expressive structure. We propose two simple but effective designs for the task and structure token generation. They could be easily replaced depending on the realistic applications on hand.

Task Token Generation Motivated by the continuous token representation in NLP, the task token $T_{\text{task}}(y_c)$ is embedded into a trainable vector: $e_c = T_{\text{task}}(y_c) \in \mathbb{R}^d$. For the total C classes in the downstream node classification, the task token embeddings are defined by: $E = [e_1, \cdots, e_C]^\top \in \mathbb{R}^{C \times d}$. The task token could be understood as a class-prototype node added to the original graph, where the node classification is performed by querying every class-prototype node. The optimal embedding of task token $T_{\text{task}}(y_c)$ should be at the center of node embeddings of class y_c.

Existing prompt-based learning methods often design the common tokens shared by all training samples [164, 165]. However, it would be hard for the distinct nodes over graph to use and tune the single task token embeddings E. In real-world networks, one of the inherent graph characteristics is cluster structure, and each node belongs to one cluster. Nodes within the same cluster have the dense connections to each other, while they are sparsely linked to the nodes from other clusters. Given the edge prediction pretext task, the pre-trained node embeddings will also be clustered in the embedding space. As explained before, the optimal embedding of task token $T_{\text{task}}(y_c)$ should thus varies with clusters. To better conduct the node classification job at each cluster, we propose the cluster-based task token generation consisting of three steps:

- First, as shown in Figure 7.1, we adopt the scalable clustering module (e.g., METIS [119]) to pre-process and split nodes into multiple non-overlapped

clusters: $\{\mathcal{G}_1, \cdots, \mathcal{G}_M\}$, where M is the hyper-parameter of cluster number.

- Second, for each cluster m, we train an independent task token embeddings: $E^m = [e_1^m, \cdots, e_C^m]^\top \in \mathbb{R}^{C \times d}$.

- Third, given task token $T_{\text{task}}(y_c)$ of node v_i at cluster m, it is embedded by vector e_c^m.

Structure Token Generation Instead of exclusively using target node v_i for the downstream classification, we apply structure token $T_{\text{str}}(v_i)$ to leverage the expressive neighborhood information. According to the social influence theory, the proximal nodes tend to possess the similar feature attributes and class patterns. One would be much easier to classify a node by taking into account the positively relative patterns, which also provide the redundant information to make the classification decision robust. Structure token $T_{\text{str}}(v_i)$ denotes the subgraph centered at node v_i. Herein we leverage the first-order adjacent nodes, the most simple subgraph, to formulate $T_{\text{str}}(v_i)$. We then embed structure token $T_{\text{str}}(v_i)$ to continuous vector as:

$$e_{v_i} = a_i * h_i + \sum_{v_j \in \mathcal{N}(v_i)} a_j * h_j. \tag{7.5}$$

a_i is the weight to aggregate the neighborhood representations, and is learned from attention function. Note that the prompting function in NLP [163] relies on the defined semantic template to inform the downstream task. Comparing with the textual language template, we define the structure token within the graph prompt by the informative neighbors of target nodes.

7.3.3 Prompt Initialization and Orthogonal Prompt Constraint

Regarding the task token embeddings $E^m = [e_1^m, \cdots, e_C^m]^\top$ at cluster m, the simplest initialization way is to use random initialization, and then train from scratch. However, such vanilla initialization may deteriorate the node classification at the initial training stage, since the optimal task token embeddings should be at the center of node representations. Conceptually, our prompt-based transfer learning paradigm uses the pre-trained GNNs as a good initialization. It follows that pre-trained node representations might serve as good initialization spots. Therefore, we initialize token embedding e_c^m by the mean of node representations, which are given by the training nodes of class y_c at cluster m. This means initialization provides the valid task tokens, and ensures the correct classification at the initial stage.

To conduct correct node classification, the prime condition is the task token embeddings of different classes should be irrelevant to each other. We utilize orthogonal constraint to keep the orthogonality of task token embeddings during model fine-tuning:

$$\mathcal{L}_o = \sum_m \|E^m (E^m)^\top - I\|_F^2. \tag{7.6}$$

7.3.4 Overall Learning Process

As mentioned in Section 7.3.1, the transfer learning framework of "pre-train, prompt, finee-tune" contains three steps, namely prompt addition, prompt answer, and prompt tuning. Based on the designs of graph prompting function and orthogonal prompt constraint, we reiterate the learning process. First, the prompt addition step modifies target node v_i with token pair $[T_{\text{task}}(y), T_{\text{srt}}(v_i)]$, and represents them with embeddings $[e_c^m, e_{v_i}]$. Second, the prompt answer evaluates a series of token pairs for the different classes, and classifies a node by task token accompanied with the highest linking

probability. The prompt tuning optimizes the pre-trained GNNs and token embeddings as below:

$$\begin{aligned} \min_{\theta,\phi,E^1,\cdots,E^M} \quad & \sum_{(v_i,y_c)} \mathcal{L}^{\text{pre}}(p_\phi^{\text{pre}}(e_c^m, e_{v_i}); g(y_c, v_i)) + \lambda \mathcal{L}_o, \\ \text{s.t.} \quad & \theta^{\text{init}} = \theta^{\text{pre}}, \phi^{\text{init}} = \phi^{\text{pre}}. \end{aligned} \tag{7.7}$$

λ is the loss hyper-parameter. ϕ^{pre} is the pre-trained parameters of projection head according to Eq. (7.2).

Why prompting in GNNs. Besides the benefit of bridging the task gap, the graph prompting function has more other advantages, including tuning efficiency, informative template for downstream applications, and easy deployment. (I) Tuning efficiency. Compared with the traditional transfer learning, the pre-trained GNNs and projection head are both reused without introducing the new projection head. The pre-trained knowledge could be elicited directly to accelerate the convergence in the downstream task. We provide extensive experiments below to demonstrate the efficiency of our GPPT. (II) Informative template. The graph prompting function provides the flexible template to include the informative signals for the downstream task. The task and structure token can be tailored to boost each specific case. (III) Easy deployment. As verified by the following studies, the pre-trained model shows promising results with only a few tuning epochs or even without fine-tuning. One only needs to carefully update the graph prompting function. This property is much preferred in the large-scale graph, where the training of GNNs requires costly computation and memory. We are able to fix GNNs once they are pre-trained, and tune prompt efficiently and scalablely similar to MLP for each new application.

7.4 Evaluation

We experiment in the node classification task on benchmark datasets with the goal of answering the following research questions. **Q1:** Compared with supervised training, joint optimization and pre-training baselines, how effective is GPPT to boost the node classification? **Q2:** How efficient is the graph prompting function to adapt the pre-trained model for the downstream application with only a few epochs or even without fine-tuning? **Q3:** How does GPPT perform in the few-shot setting?

7.4.1 Experimental Setup

Dataset. We evaluate the proposed framework GPPT on eight popular benchmark datasets, including citation networks [166] (i.e, Cora, Citeseer, Pubmed), Reddit [145], CoraFull [167], Amazon-Co-Buy [42] (i.e., Computer and Photo), ogbn-arxiv [168].

Table 7.1 : Node classification accuracy in percent. The best case is in bold. Imp(%) at the last column means the average improvement of GPPT over baseline.

Training schemes	Methods	Data								Imp(%)
		Cora	**Citeseer**	**Pubmed**	**CoraFull**	**Computer**	**Photo**	**ogbn-arxiv**	**Reddit**	
Supervised	GraphSAGE [145]	81.07±0.30	67.95±0.34	77.24±0.62	65.86±1.73	86.28±0.27	90.95±0.98	65.07±0.67	91.25±0.14	1.60
	GCN [166]	76.32±0.95	65.40±0.13	75.33±0.27	62.79±0.41	84.29±0.61	90.33±0.70	64.26±0.15	87.39±0.28	4.98
	GAT [169]	78.47±1.68	65.61±0.57	75.91±0.14	63.20±0.77	83.95±0.92	88.93±0.50	64.31±1.06	88.00±0.72	4.46
Joint	GAE [170]	78.87±1.22	66.23±0.77	76.37±0.41	65.16±1.30	73.30±0.29	85.29±0.65	65.62±0.44	91.16±1.29	5.63
	Joint-Edge [154]	80.59±1.13	69.11±1.28	78.99±0.52	66.40±0.28	**88.57**±0.69	89.79±0.44	65.51±0.17	91.60±0.16	0.79
	Joint-DGI [171]	78.36±0.38	68.47±0.82	76.66±0.11	64.93±0.57	85.09±0.14	90.38±0.14	65.60±0.28	91.22±0.75	2.40
Pre-train	Pre-Edge [154]	81.00±0.93	67.39±0.98	78.51±0.14	65.62±1.71	86.41±1.46	90.35±0.17	65.59±0.18	91.38±0.16	1.50
	Pre-DGI [171]	76.25±0.90	65.09±1.14	76.42±1.61	63.60±0.73	85.34±0.94	90.94±0.34	65.00±0.18	87.96±0.33	4.23
Prompt	GPPT(w/o C)	80.90±0.75	67.64±0.21	79.28±1.04	65.81±1.33	86.57±0.75	91.58±0.16	65.66±0.88	91.02±0.57	1.16
	GPPT(w/o N)	81.16±0.30	67.74±1.74	79.58±1.43	66.03±1.11	86.95±0.19	**92.26**±0.60	65.96±0.14	**92.51**±0.66	0.57
	GPPT	**81.40**±0.48	**69.21**±0.48	**79.73**±0.38	**66.91**±1.09	87.38±0.85	92.13±1.01	**65.94**±0.24	92.45±0.59	-

Table 7.2 : Test accuracies (in percent) of pre-trained models without fine-tuning. The higher one has better adaptability.

Training schemes	Methods	Data								Imp(%)
		Cora	**Citeseer**	**Pubmed**	**CoraFull**	**Computer**	**Photo**	**ogbn-arxiv**	**Reddit**	
Pre-train	Pre-Edge	46.68±0.39	23.96±0.94	21.96±0.26	21.75±0.24	45.84±0.79	42.69±0.13	38.59±0.22	88.72±0.33	101.26
	Pre-DGI	5.96±0.34	16.95±0.85	40.75±0.18	5.76±0.13	37.26±0.99	22.30±0.25	9.16±0.48	65.39±0.11	396.85
Prompt	GPPT(fix)	**77.19**±0.31	**63.84**±0.45	**78.90**±0.18	**41.36**±0.44	**77.27**±0.25	**87.15**±0.14	**60.22**±0.67	**88.77**±0.42	-

Approaches. To evaluate the proposed "pre-train, prompt, fine-tune" learning framework of GPPT, we compare with state-of-the-art training schemes, including supervised learning, joint optimization, and pre-training. They are introduced below:

- **Supervised learning:** It uses the training nodes to learn GNNs, which are directly applied for model inference. We consider the following widely-studied GNN baselines, including GraphSAGE(GS) [145], GCN [166], and GAT [169].

- **Joint optimization:** By simply combining the self-supervised learning objective with downstream objective, the joint learning updates GNNs to optimize both tasks. GAE [170] uses graph auto-encoder, and jointly optimizes the node classification loss with refactoring loss. We further apply GraphSAGE, the most popular model, as backbone and consider two joint optimization methods: EdgeMask [154] learning the masked edge prediction and DGI [171] leveraging the contrastive learning of deep graph infomax. They have been generally adopted to learn the graph structure. To separate from the following pre-training setting, we use Joint-Edge and Joint-DGI to denote the joint optimization.

- **Pre-training:** Following the transfer learning strategy of "pre-train, fine-tune", the pre-trained models are fine-tuned in the downstream applications. Similar to

Table 7.3 : Test accuracy in percent of GPPT by ablating prompt initialization (init.) or orthogonal prompt constraint (cons.). The ↓/↑ arrow means the decreasing/improvement compared with the corresponding GPPT in Table 7.1.

Methods	**Cora**	**Citeseer**	**Pubmed**	**CoraFull**	**Computer**	**Photo**	**ogbn-arxiv**	**Reddit**
GPPT(w/o cons.)	80.81±0.14(↓)	67.73±0.90(↓)	78.72±0.26(↓)	65.11±0.67(↓)	86.82±0.59(↓)	91.71±0.98(↓)	64.82±0.76(↓)	92.10±0.32(↓)
GPPT(w/o init.)	79.30±0.10(↓)	67.01±0.08(↓)	77.82±0.08(↓)	66.15±0.07(↓)	87.07±0.12(↓)	91.70±0.04(↓)	66.21±0.22(↑)	92.53±0.15(↑)

the joint optimization, we compare with Pre-Edge and Pre-DGI, where EdgeMask DGI are instead applied as pretext tasks, respectively.

- **Prompt-based tuning:** Our GPPT is realized by following the learning paradigm of "pre-train, prompt, fine-tune". Meanwhile, two simple versions are developed: GPPT (w/o C) and GPPT (w/o N). Specifically, GPPT (w/o C) removes the graph clustering and applies a single task token matrix; GPPT (w/o N) replaces the structure token with target node itself to ablate the neighborhoods during classification.

Implementations Following the previous efforts and ensuring the fair comparison, we adopt the layer number of 2 and the hidden units of 128 for all GNNs. The Adam optimizer accompanied with weight decay of 5e-4 is applied at both the pre-training and fine-tuning stages. The learning rate is chosen from [0.001,0.005] depending on datasets. For our GPPT, we use the orthogonality prompt constraint coefficient λ of 0.01. The clustering numbers in Cora, Citeseer, Pubmed, CoraFull, Computer, Photo, ogbn-arxiv, Reddit are 7, 6, 3, 5, 10, 8, 10, 10, respectively.

7.4.2 Performance Analysis

Node Classification Studies. To answer research question **Q1**, we list the comparison between different categories of training methods in Table 7.1. We make the following major observations.

❶ *Prompt-based learning methods generally obtain the best performance on the benchmarks.* While most of joint optimization methods outperform the supervised learning by leveraging the self-supervised structure signals, the pre-training based methods tend to damage the downstream node classification. As analyzed before, there exists the significant training objective gap between pretext and downstream tasks. Under the paradigm of "pre-train, fine-tune", the pre-trained knowledge may be gradually filtered out during the long fine-tuning process. The pre-training thus fails to server as good initialization for the downstream application, and results in the comparable or even worse classification accuracies. For example, Pre-DGI has 3.55% dropping compared with supervised learning model GraphSAGE on Cora, which means the negative transfer. With the graph prompting function, GPPT directly bridges the task gap to guarantee the effectiveness of pre-trained model in the downstream applications. Comparing with all the other training schemes, the prompt-based learning shows the powerful capability to elicit the pre-trained graph knowledge. The average improvement over the baselines could be up to 3.19%.

❷ *The graph clustering and neighborhood structure are crucial for the informative prompt token designs.* By ablating any of them, it is observed that the generalization performance is generally decreased. The experimental results are in line with our motivations. First, since the node embeddings are pre-trained to memorize the cluster structure, the separating task tokens (which representing the label prototypes) should be adopted for different clusters to better conduct the node classification. The more

studies of cluster number are provided in the following experiments. Second, the local neighborhood structure is informative for the node classification. The proximal nodes tend to possess the similar feature attributes and class patterns. One would be much easier to classify a node by taking into account the positively relative patterns, which also provide the redundant information to make the classification decision robust. In the future work, the more task and structure tokens need to be tailored for the specific downstream applications.

Fine-tuning Efficiency Analysis. Compared with other training strategies, the prompt could efficiently adapt the pre-trained model to downstream applications with only a few fine-tuning steps or even without fine-tuning. To answer **Q2**, we compare the training loss dynamics between GraphSAGE, Joint-Edge, Pre-Edge, and GPPT on Cora and Citeseer in Figure 7.2. Furthermore, given the pre-trained models in pre-training based methods and prompt-learning based approaches, we fix them and apply to the downstream tasks to compare their adaptability. The test accuracy is listed in Table 7.2. We make the following major observations.

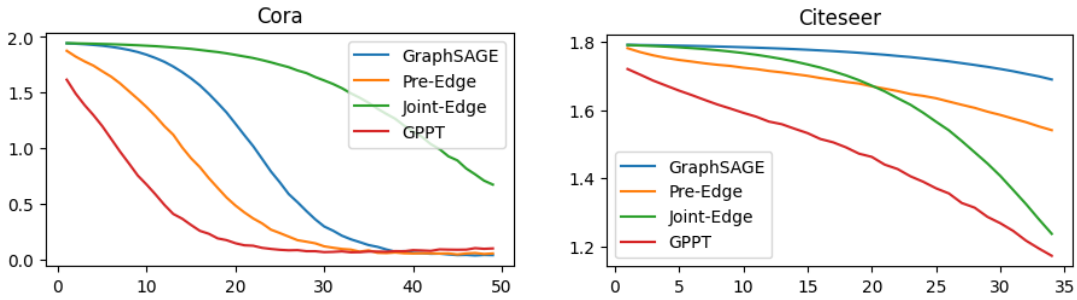

Figure 7.2 : Training losses with epochs on Cora and Citeseer.

❸ *GPPT consistently shows the fastest convergence towards the optimal downstream points.* In general, GPPT only takes 23 fine-tuning epochs to optimize the downstream

tasks, given the average convergence steps of 100 in other training strategies. GPPT speeds up the fine-tuning efficiency by up to 4.32X. This is because the graph prompting function in GPPT directly bridges the task gap, which enables the pre-trained GNNs and projection head could be applied to downstream task without any modification. Therefore, the pre-trained knowledge is directly elicited in the downstream application, which brings the high fine-tuning efficiency.

❹ *The prompt-based learning paradigm even enables the pre-trained GNNs to be transferred without any tuning, comparing with the traditional pre-training methods.* In Table 7.2, Pre-Edge and Pre-DGI are fine-tuned with one step to adapt the new node classification layer to the downstream task. In contrast, all the pre-trained parameters are fixed in GPPT since the node classification has already been reformulated to the pretext edge prediction. We only learn the graph prompting function, a small adaption module easily to be updated. It is observed that GPPT delivers the significant improvements over Pre-Edge and Pre-DGI. The pre-training based methods without careful fine-tuning have very poor performances, e.g., the only 5.96% accuracy of Pre-DGI on Cora. These observations positively validate the applicability and reliability of GPPT to real-world systems: Without conducting the message passing and training GNNs again, the pre-trained model (or node embeddings) could be applied directly to any downstream task by only tuning the prompt similar to MLP. Such efficiency is much preferred for the large-scale graph, where the training in the whole graph is time consuming and memory expensive.

Few-Shot Setting Analysis. The self-supervised pretext task provides unlabeled information to better tackle the few-shot learning problem. To answer research question **Q3**, we consider the more difficult few-shot node classification setting, by randomly

masking a portion of training labels on the concerned datasets. The test accuracy obtained with 50% masking ratio is listed in Table 7.4. We make observations:

Table 7.4 : Test accuracy (%) in few-shot learning setting with masking ratio of 50%.

Training schemes	Methods	Data								Imp(%)
		Cora	**Citeseer**	**Pubmed**	**CoraFull**	**Computer**	**Photo**	**ogbn-arxiv**	**Reddit**	
Supervised	GraphSAGE	68.40±0.96	57.98±0.98	68.23±0.46	62.72±0.47	85.04±0.94	90.60±0.81	64.56±0.75	91.07±0.86	5.10
	GCN	71.78±0.50	52.15±0.27	65.05±0.15	61.17±0.53	83.54±0.45	89.92±0.28	64.19±0.59	87.50±0.29	7.90
	GAT	74.94±1.26	59.50±0.61	69.30±0.97	61.08±0.33	83.02±0.41	87.96±0.84	63.97±0.69	87.65±0.90	4.94
Joint	GAE	73.83±0.32	63.17±0.24	66.74±0.33	63.76±0.16	83.55±0.45	89.63±0.93	64.14±0.33	89.70±0.25	3.62
	Joint-Edge	74.75±0.87	62.76±1.15	69.03±0.58	62.72±0.72	84.18±0.61	89.19±0.71	65.14±0.58	88.00±0.22	3.32
	Joint-Infomax	73.74±0.32	63.37±0.28	68.30±0.24	62.55±0.88	84.33±0.58	89.54±0.55	65.01±0.50	90.91±0.96	3.08
Pre-train	Pre-Edge	76.38±0.89	65.49±0.90	71.29±0.66	62.79±0.66	85.33±0.89	90.04±0.69	64.86±0.67	90.91±0.36	1.41
	Pre-Infomax	68.91±0.89	63.40±0.89	68.98±0.36	60.18±0.49	84.24±0.97	89.83±1.18	64.56±0.94	87.59±0.73	4.91
Prompt	GPPT(w/o C)	76.84±0.68	64.79±1.03	71.78±0.32	63.43±0.36	86.08±0.83	90.30±0.43	65.38±0.82	90.81±0.92	1.02
	GPPT(w/o N)	76.96±0.45	65.63±0.52	71.00±0.50	63.38±0.75	**86.82**±0.68	90.80±0.45	65.47±0.52	**92.44**±0.23	0.57
	GPPT	**77.16**±1.35	**65.81**±0.97	**72.23**±1.22	**64.01**±0.73	86.80±0.26	**91.40**±0.92	**66.13**±0.44	92.13±0.57	-

❺ *Supervised learning and joint optimizing methods have poor results in the few-shot setting.* The general supervised learning methods utilize empirical risk minimization. As the amount of labeled data decreases, the prior knowledge that the model can acquire will also decrease, which will lead to worse performance.

❻ *The prompt-based learning methods effectively elicit the pre-trained knowledge to improve few-shot learning.* Compared with the supervised learning and joint optimization, the pre-training approaches purely optimize the pre-text objective to capture the graph structure knowledge, without suffering from the masked training labels. The pre-trained model serves as good initialization to the following few-shot problem. Furthermore, compared with pre-training approaches, the prompt-based methods train the extra prompting function to better inform the downstream node

classification, could thus reduce the need of costly annotated data. GPPT delivers the average improvements ranging from 0.57% to 7.90%.